SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD PIPELINE DESIGN EXERCISE

Intelligent Public Health Surveillance and Disease Outbreak Prediction System

Course Code	CSA10
Course Title	Software Engineering
CO Assessed	CO4
Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
CO4 Statement	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics
Weightage	20%
Total Marks	25
Duration	60 minutes
Student Name	Sheik mohamed M
Register Number	192511145
Date	

Code of Conduct

I Sheik Mohamed M(192511145) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sheik Mohamed M

Assigned Problem Statement

19. Intelligent Public Health Surveillance and Disease Outbreak Prediction System

Industry Context

Public health agencies require continuous disease surveillance to detect outbreaks and implement preventive measures. AI, Big Data Analytics, Geographic Information Systems (GIS), IoT, and cloud-based healthcare platforms enable early detection of infectious diseases and support evidence-based public health decision-making.

Problem Statement

Disease outbreaks spread rapidly due to delayed detection, fragmented healthcare reporting systems, and inadequate epidemiological analysis. Existing surveillance systems often lack real-time monitoring and predictive capabilities. An intelligent disease surveillance platform should collect healthcare data from hospitals, laboratories, and public health agencies, analyze disease trends using AI algorithms, identify potential outbreak hotspots, and provide early warning notifications. The system should also support visualization, resource planning, and policy decision-making to minimize disease transmission and improve community health.

Objectives

- Collect real-time public health data.
- Predict disease outbreaks using AI.
- Identify outbreak hotspots through GIS.
- Generate early warning alerts.
- Monitor disease transmission trends.
- Support healthcare resource planning.
- Provide epidemiological analytics dashboards.
- Improve public health decision-making.

Expected Outcomes

- Early disease outbreak detection.
- Improved public health surveillance.
- Reduced disease transmission.
- Better healthcare resource allocation.
- Faster emergency response.
- Enhanced epidemiological analysis.
- Improved healthcare planning.
- Increased community health protection.

1. Introduction and CI/CD Requirement Analysis

This document presents the design of a Continuous Integration and Continuous Deployment (CI/CD) pipeline for the Intelligent Public Health Surveillance and Disease Outbreak Prediction System. The system is composed of several independently deployable microservices — data ingestion, AI-based outbreak prediction, GIS hotspot mapping, early-warning alerting, epidemiological dashboards, and resource planning — that must be built, tested, secured, and released reliably and frequently as public health requirements evolve.

Because the system directly supports time-critical outbreak detection and alerting, the pipeline must automate integration and testing to catch defects early, enforce security and data-privacy checks given the sensitivity of health data, and support rapid, low-risk releases with the ability to roll back instantly if a deployment degrades alerting accuracy or availability. The functional and non-functional CI/CD requirements below were derived from these needs.

1.1 CI/CD Functional Requirements

ID	Requirement Description
CR-01	The pipeline shall automatically trigger on every code commit or pull request to the source repository.
CR-02	The pipeline shall automatically build each affected microservice (data ingestion, AI prediction engine, GIS service, alerting service, dashboard, resource-planning service).
CR-03	The pipeline shall execute automated unit and integration tests for every affected module before merge.
CR-04	The pipeline shall perform static code analysis and enforce a code-quality gate prior to build promotion.
CR-05	The pipeline shall perform automated security scanning (SAST, dependency, and container image vulnerability scanning) on every build.
CR-06	The pipeline shall package validated builds into versioned, deployable container artifacts.
CR-07	The pipeline shall automatically deploy build artifacts to a staging/test environment for further validation.
CR-08	The pipeline shall execute automated acceptance and performance tests in staging before promotion to production.
CR-09	The pipeline shall enforce a manual approval gate before deployment of critical services (AI prediction engine, alerting service) to production.

ID	Requirement Description
CR-10	The pipeline shall deploy approved artifacts to production using a zero/low-downtime release strategy.
CR-11	The pipeline shall send notifications to the DevOps/release team on success or failure at every stage.
CR-12	The pipeline shall support automatic rollback to the last stable release on deployment failure or critical health-check alert.
CR-13	The pipeline shall maintain versioned build artifacts and a deployment history for traceability and audit.
CR-14	The pipeline shall retrieve secrets and credentials from a secure secrets manager rather than storing them in plain text.
CR-15	The pipeline shall generate execution reports (test results, coverage, security scan summary) accessible to the development team.

1.2 CI/CD Non-Functional Requirements

ID	Category	Description
NCR-01	Performance	The complete pipeline, from commit to staging deployment, shall complete within 15 minutes for a standard change.
NCR-02	Reliability	Transient infrastructure failures shall trigger automatic retries, minimizing false pipeline failures.
NCR-03	Security	The pipeline shall enforce least-privilege access to deployment credentials and target environments.
NCR-04	Scalability	The pipeline shall support parallel builds and tests across independent microservices.
NCR-05	Availability	The CI/CD tooling infrastructure shall maintain high availability to avoid blocking development.
NCR-06	Auditability	Every pipeline run shall be logged with timestamp, triggering commit/user, and outcome.
NCR-07	Maintainability	Pipeline configuration shall be defined as code, version-controlled, and reviewed through pull requests.
NCR-08	Compliance	The pipeline shall prevent real patient data from being used in non-production environments, per data-protection policy.

2. Pipeline Architecture and Workflow

The pipeline is organized into three logical phases: Continuous Integration (source management through packaging), Continuous Deployment to Staging (automated validation in a production-like environment), and Production Release with Monitoring and Rollback. Each phase gates the next — a build only proceeds to the following stage if all checks in the current stage pass, ensuring that only validated, secure artifacts reach production.

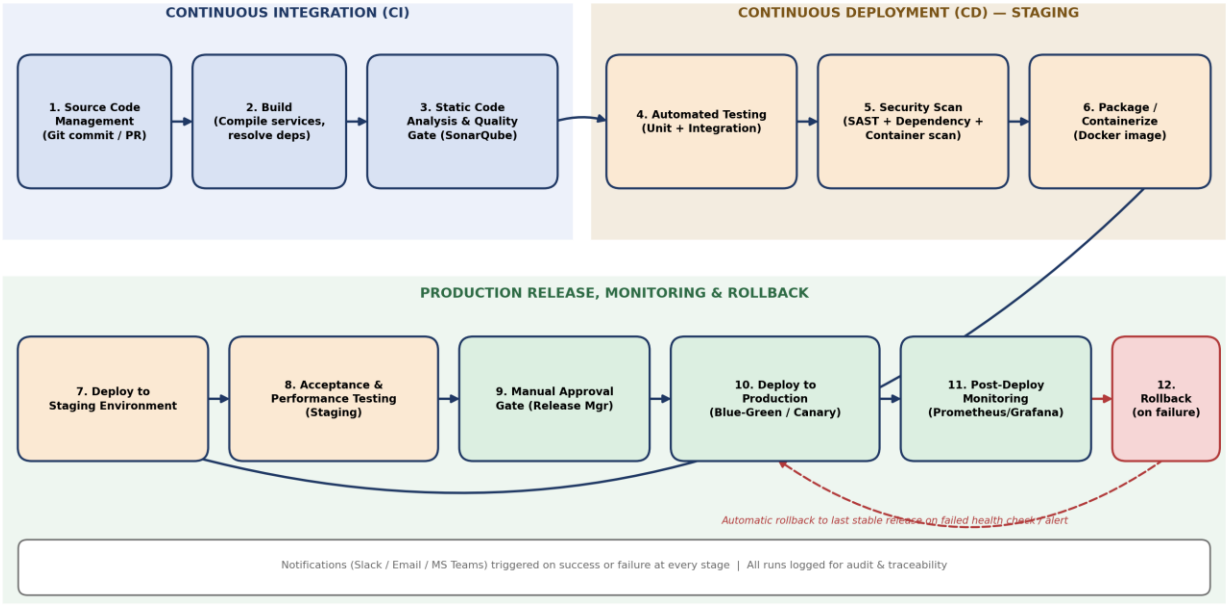

Figure 1: End-to-end CI/CD pipeline for the Disease Outbreak Prediction System, from code commit to production release with monitoring and automatic rollback.

A commit or pull request triggers the CI phase (Stages 1–6), which builds, quality-checks, tests, security-scans, and packages the affected microservice(s). The packaged image is then automatically deployed to staging (Stages 7–8) for acceptance and performance validation. A manual approval gate (Stage 9) precedes production release (Stage 10), which uses a blue-green or canary strategy to minimize risk. Post-deployment monitoring (Stage 11) continuously checks service health, and the pipeline automatically rolls back to the last stable release (Stage 12) if failures are detected. Notifications and audit logging run throughout every stage.

3. Pipeline Stages – Detailed Description

Stage	Purpose	Key Actions	Trigger	Tools (Indicative)	Succe
1. Source Code Management	Version-control application code and coordinate changes across teams.	Developer commits/pushes to a feature branch; opens a pull request against 'develop'.	git push / pull request	Git, GitHub/GitLab, branch protection rules	PR op passin checks review before
2. Build	Compile and assemble each microservice with its dependencies.	Resolve dependencies; compile backend services; bundle dashboard frontend.	On PR / on merge to develop	Maven/Gradle, npm/pip build scripts, GitHub Actions/Jenkins runners	Build with n compi errors.
3. Static Code Analysis & Quality Gate	Detect code smells, complexity, and maintainability issues early.	Run static analyzer; compare against quality gate thresholds.	After successful build	SonarQube / SonarCloud	Qualit passed blocke issues
4. Automated Testing (Unit + Integration)	Verify functional correctness of individual modules and their interactions.	Execute unit test suites per service; run integration tests against mocked/test dependencies.	After quality gate passes	JUnit/pytest, Testcontainers, Postman/Newman	All tes code c 80%.
5. Security Scanning	Identify vulnerable dependencies, insecure code patterns, and container CVEs before release.	Run SAST scan on source; scan dependency manifest; scan built container image.	After tests pass	Snyk / OWASP Dependency-Check, Trivy	No cri high-s vulner open.
6. Packaging / Containerization	Produce a consistent, versioned deployable artifact.	Build Docker image; tag with semantic version and commit SHA; push to registry.	After security scan passes	Docker, private container registry	Image tagged pushed succes
7. Deploy to Staging	Validate the release in an environment	Deploy container image to the staging Kubernetes	Automatic, after packaging	Kubernetes, Helm	Deplo health probes

Stage	Purpose	Key Actions	Trigger	Tools (Indicative)	Succe
	resembling production.	namespace using Helm.			
8. Acceptance & Performance Testing	Confirm end-to-end functionality and system behavior under load.	Run automated UI/acceptance tests for key journeys (alerts, dashboards); run load tests simulating concurrent facility nodes.	After staging deploy	Selenium/Cypress, JMeter/k6	All cri journe respor within target
9. Manual Approval Gate	Add a human checkpoint before affecting real users/health officials.	Release manager reviews staging results and approves or rejects promotion.	After staging tests pass	GitHub Actions environments / Jenkins input step	Explic record proce
10. Deploy to Production	Release the validated build to end users with minimal risk.	Roll out using blue-green or canary strategy; shift traffic gradually.	After manual approval	Kubernetes, Helm, service mesh/ingress traffic splitting	New v servin with n spike.
11. Post-Deployment Monitoring	Continuously verify production health after release.	Monitor error rates, latency, and health-check endpoints in real time.	Continuous, post-deploy	Prometheus, Grafana, Alertmanager	Metric thresh active alerts.
12. Rollback (on failure)	Restore service quickly if the new release is unhealthy.	Automatically revert to the last stable release if health checks fail or error-rate/alert thresholds are breached.	Triggered by monitoring alert	Helm rollback / kubectl rollout undo	Previc versio incide and nc

4. Tools and Technology Selection

The tools below were selected to match the system's polyglot microservice architecture, its need for rapid but safe releases, and the heightened security/privacy requirements of handling public-health data.

Stage / Category	Selected Tool(s)	Justification
Source Code Management	Git + GitHub/GitLab	Industry-standard distributed version control with pull-request review workflows and native CI integration.
CI/CD Orchestrator	GitHub Actions (or Jenkins)	Pipeline-as-code with a large ecosystem of actions/plugins; supports parallel jobs, ideal for a multi-microservice system.
Build Automation	Maven/Gradle (Java services), npm/pip (frontend/AI scripts)	Matches the language stack of each service; mature, well-documented dependency and build management.
Static Code Analysis	SonarQube	Combines code smells, complexity, duplication, and security hotspot detection with configurable quality gates.
Unit / Integration Testing	JUnit/pytest, Testcontainers	Enables isolated, repeatable tests against containerized dependencies (e.g., mock hospital API, test database).
Security Scanning	Snyk / OWASP Dependency-Check + Trivy	Detects vulnerable open-source dependencies and container image CVEs — critical given the system handles sensitive health data.
Packaging	Docker + private container registry	Produces a consistent, portable artifact that behaves identically across staging and production.
Deployment / Orchestration	Kubernetes + Helm	Declarative deployments, built-in rolling/blue-green/canary strategies, auto-scaling for outbreak-driven traffic surges.

Stage / Category	Selected Tool(s)	Justification
Secrets Management	HashiCorp Vault / cloud-native secrets manager	Centralizes credentials (DB, API keys) with access policies, avoiding hardcoded secrets in pipeline scripts.
Monitoring & Alerting	Prometheus + Grafana + Alertmanager	Real-time metrics collection, dashboards, and alert routing to detect degraded service health quickly.
Notifications	Slack / Email / MS Teams webhooks	Immediate visibility into pipeline outcomes for the DevOps and health-IT teams.
Performance Testing	JMeter / k6	Simulates concurrent facility/IoT load to validate NFR performance targets before production release.

5. Automated Testing Strategy

Multiple layers of automated testing are integrated at different pipeline stages to catch defects as early and cheaply as possible, while still validating end-to-end behavior and performance before production release.

Test Level	Scope	Tool / Framework	Pipeline Stage	Pass Criteria
Unit Testing	Individual functions/modules, e.g., risk-score calculation, validation rules.	JUnit / pytest	Build	All unit tests pass; coverage ≥ 80% per service.
Integration Testing	Interactions between services, e.g., data ingestion → AI prediction engine.	Testcontainers, Postman/Newman	Automated Testing	All integration endpoints return expected responses; no contract breakages.
Static Analysis / Code Quality	Code smells, complexity, duplication, security hotspots.	SonarQube	Quality Gate	Quality gate passed; 0 blocker/critical issues.
Security Testing (SAST/SCA)	Source code and dependency vulnerability scanning.	Snyk / OWASP Dependency-Check	Security Scan	No critical/high vulnerabilities open.
Container Security Scanning	Base image and installed package CVEs.	Trivy	Security Scan	No critical CVEs in the built image.
Acceptance Testing	End-to-end user journeys (alert generation, dashboard filtering, login).	Selenium / Cypress	Staging	All critical user journeys pass.
Performance Testing	API and dashboard response under concurrent load.	JMeter / k6	Staging	Response time within 5s under 500 concurrent nodes (NFR-

Test Level	Scope	Tool / Framework	Pipeline Stage	Pass Criteria
				01/NCR target).
Smoke Testing (Production)	Basic health checks immediately after deployment.	Custom health-check scripts	Post-Deploy	All services report healthy status within 2 minutes of deploy.

6. Deployment Strategy and Environments

Environment	Purpose	Characteristics
Development	Used by developers for feature work and early testing.	Frequently updated on every commit; uses synthetic data; minimal access restrictions; unstable by design.
Testing / Staging	Mirrors production configuration to validate releases before go-live.	Deployed automatically after packaging; uses masked/synthetic health data; runs acceptance and performance tests.
Production	Serves real public health officials and live surveillance data.	Deployed only after manual approval; strict access control, encryption, and monitoring; blue-green/canary rollout.

6.1 Production Release Strategy

Production deployments use a Blue-Green strategy for routine releases: a new ('green') environment is deployed alongside the current ('blue') production environment, validated via automated smoke tests, and traffic is switched over only once healthy — giving an instant rollback path by simply routing traffic back to blue.

For updates to the AI prediction model specifically, a Canary strategy is used instead: the new model version initially serves a small percentage of live traffic (e.g., 5–10%) so its prediction accuracy and alerting behavior can be validated against real data before full rollout, reducing the risk of a faulty model triggering false or missed outbreak alerts.

7. Security, Quality Checks, Notifications and Rollback

7.1 Security Integration (DevSecOps)

- Static Application Security Testing (SAST) scans source code for insecure patterns on every build.
- Software Composition Analysis (SCA) scans third-party dependencies for known CVEs before packaging.
- Container image scanning checks base images and installed packages for vulnerabilities prior to deployment.
- Secrets (database credentials, API keys) are injected at deploy time from a secrets manager — never committed to source control or pipeline scripts.
- Role-based access control restricts who can approve production deployments and who can access production secrets.
- Data-handling checks prevent real patient records from being used to seed non-production environments, per NCR-08.

7.2 Quality Checks

- A SonarQube quality gate blocks promotion on blocker/critical code issues or insufficient test coverage.
- Automated linting enforces consistent coding standards across all microservices.
- Performance budgets (response-time thresholds) are enforced in the staging load-test stage before production promotion.

7.3 Notifications

- Slack/Email/MS Teams alerts are sent to the DevOps channel on the start, success, or failure of each pipeline stage.
- Critical failures (e.g., security scan block, failed production health check) additionally page the on-call engineer.

7.4 Rollback Mechanism

- Post-deployment monitoring continuously evaluates error rate, latency, and health-check endpoints against thresholds.
- If thresholds are breached within the post-deploy observation window, the pipeline automatically triggers 'helm rollback' / 'kubectl rollout undo' to restore the last known-good release.
- A rollback event automatically raises an incident ticket and notifies the release manager and on-call engineer.

8. Sample Pipeline Configuration (Illustrative)

The following simplified, illustrative pipeline-as-code configuration (GitHub Actions style) shows how the stages above map to executable jobs for one microservice (the outbreak-prediction service). Each of the other microservices follows an equivalent workflow file.

```
name: outbreak-prediction-service-pipeline

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build service
        run: ./gradlew build
      - name: Static code analysis
        run: sonar-scanner -Dsonar.qualitygate.wait=true
      - name: Unit & integration tests
        run: ./gradlew test integrationTest
      - name: Dependency & container scan
        run: |
          snyk test --severity-threshold=high
          trivy image outbreak-prediction:${{ github.sha }}

  package-deploy-staging:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - name: Build & push image
        run: |
          docker build -t registry/outbreak-prediction:${{ github.sha }} .
          docker push registry/outbreak-prediction:${{ github.sha }}
      - name: Deploy to staging
        run: helm upgrade --install outbreak-svc ./chart -f values-staging.yaml
```

```
    - name: Acceptance & performance tests
      run: |
        cypress run --env baseUrl=$STAGING_URL
        k6 run perf/load-test.js

deploy-production:
  needs: package-deploy-staging
  environment:
    name: production
    # requires manual approval configured on this GitHub environment
  runs-on: ubuntu-latest
  steps:
    - name: Blue-green deploy to production
      run: helm upgrade --install outbreak-svc ./chart \
             -f values-prod.yaml --set strategy=blue-green
    - name: Post-deploy health check
      run: ./scripts/health-check.sh --rollback-on-failure
```

9. Branching Strategy and Version Control Workflow

A Git Flow–based branching model governs how code moves through the pipeline, ensuring that only reviewed, tested code reaches shared environments and that production changes remain traceable to a specific commit and approval.

Branch	Purpose	Pipeline Interaction
main	Always reflects the current production release.	Protected; only updated via approved release merges from 'develop' after staging validation. Triggers production deployment jobs.
develop	Integration branch for completed features awaiting release.	Every merge triggers the full CI phase (build, quality gate, tests, security scan) and staging deployment.
feature/*	Individual feature or bug-fix work by a developer.	Pull requests into 'develop' trigger build, static analysis, and unit/integration tests as pre-merge checks.
release/*	Stabilization branch cut from 'develop' ahead of a planned release.	Used for final regression/performance testing in staging before merging to 'main'.
hotfix/*	Urgent fixes applied directly against production.	Triggers an expedited pipeline path (build → security scan → staging smoke test → approval → production) for critical issues such as a broken alerting service.

10. Pipeline Metrics and DevOps KPIs

The following DevOps Research and Assessment (DORA)–aligned metrics, along with test-coverage and pipeline-stability indicators, are tracked to continuously evaluate and improve the pipeline's effectiveness for this outbreak-prediction system.

Metric	Definition	Target	Why It Matters Here
Deployment Frequency	How often the team successfully releases to production.	Multiple times per week	Indicates the pipeline enables small, frequent, lower-risk releases rather than large infrequent ones.
Lead Time for Changes	Time from code commit to running in production.	Under 1 day for standard changes	Reflects how quickly a fix (e.g., to the alerting logic) can reach health officials.
Change Failure Rate	Percentage of production deployments causing a failure or requiring rollback.	Below 10%	Measures release quality; high values indicate insufficient testing/quality gates.
Mean Time to Recovery (MTTR)	Average time to restore service after a production incident.	Under 15 minutes	Validates the effectiveness of automated monitoring and rollback (Stages 11–12).
Test Coverage	Percentage of code exercised by automated tests.	≥ 80% per service	Ensures the automated test suite provides meaningful defect-detection confidence.
Pipeline Success Rate	Percentage of pipeline runs completing without failure.	≥ 95%	Low values suggest flaky tests or unstable infrastructure needing attention.

11. Assumptions and Constraints

11.1 Assumptions

- Each microservice (data ingestion, AI prediction, GIS, alerting, dashboard, resource planning) has its own repository and pipeline definition, following the same staged workflow.
- A Kubernetes cluster with separate staging and production namespaces/clusters is available to the DevOps team.
- Staging environment data is synthetic or masked, never real patient data, per data-protection policy (NCR-08).
- The AI prediction model's training/retraining pipeline is managed separately from this application deployment pipeline, but the resulting model artifact is deployed through it.

11.2 Constraints

- Pipeline design is scoped to the 60-minute assessment; exhaustive infrastructure-as-code and multi-region disaster-recovery pipelines are out of scope.
- Tool choices are indicative and would be finalized based on the institution's existing DevOps toolchain and budget.
- Manual approval before production is mandatory for this system given its public-health impact, which slightly increases lead time compared to fully automated deployment.

12. Pipeline Risk Assessment and Mitigation

Risk	Likelihood	Impact	Mitigation
Flaky automated tests causing false pipeline failures	Medium	Medium	Quarantine flaky tests, track failure patterns, enforce a stability SLA before re-enabling them as release blockers.
Security scan blocking release due to an unpatched dependency with no fix available	Low	High	Document a risk-acceptance/exception process reviewed by a security lead, with compensating controls.
Faulty AI model version degrading outbreak-prediction accuracy after release	Medium	High	Use canary rollout for model updates; monitor prediction accuracy metrics and auto-rollback on drift.
Secrets leakage through misconfigured pipeline logs	Low	High	Mask secrets in CI logs; restrict secret scope per environment; rotate credentials regularly.
Staging environment drifting from production configuration	Medium	Medium	Manage environment configuration as code (Helm values per environment) reviewed alongside application code.

13. Requirement Traceability Matrix

The matrix below maps each CI/CD functional and non-functional requirement to the pipeline stage(s) that fulfil it.

Requirement ID	Requirement Description	Fulfilling Pipeline Stage(s)
CR-01	Trigger on commit/PR	Stage 1 – Source Code Management
CR-02	Automated build	Stage 2 – Build
CR-03	Automated unit/integration tests	Stage 4 – Automated Testing
CR-04	Static analysis/quality gate	Stage 3 – Static Code Analysis
CR-05	Security scanning	Stage 5 – Security Scanning
CR-06	Packaging into artifacts	Stage 6 – Packaging/Containerization
CR-07	Auto-deploy to staging	Stage 7 – Deploy to Staging
CR-08	Acceptance/performance tests	Stage 8 – Acceptance & Performance Testing
CR-09	Manual approval gate	Stage 9 – Manual Approval Gate
CR-10	Low-downtime production deploy	Stage 10 – Deploy to Production
CR-11	Stage notifications	All stages – Notification bar
CR-12	Automatic rollback	Stage 12 – Rollback
CR-13	Versioned artifacts/history	Stage 6 – Packaging; Stage 10 – Deploy
CR-14	Secure secrets handling	Stage 7 & 10 – Deployment (Vault integration)
CR-15	Execution reports	Stage 4, 5 – Testing & Security reports
NCR-01	Pipeline duration ≤ 15 min	Stages 1–7 (CI + staging deploy)
NCR-02	Retry on transient failure	CI orchestrator retry policy (all stages)
NCR-03	Least-privilege access	Secrets management, Stage 7 & 10
NCR-04	Parallel microservice builds	Stage 2 – Build (parallel jobs)
NCR-05	CI/CD tooling availability	CI orchestrator infrastructure
NCR-06	Run logging/audit	All stages – Notification & logging bar

Requirement ID	Requirement Description	Fulfilling Pipeline Stage(s)
NCR-07	Pipeline as code	Stage 1 – Source Code Management (pipeline YAML in repo)
NCR-08	No real patient data in staging	Stage 7 – Deploy to Staging (data masking policy)

14. Conclusion

The CI/CD pipeline designed in this document automates the full lifecycle of the Intelligent Public Health Surveillance and Disease Outbreak Prediction System — from source code commit through build, static analysis, automated testing, security scanning, packaging, staged validation, and production release with blue-green/canary deployment. Integrated quality gates, security scans, notifications, monitoring, and automatic rollback ensure that releases affecting outbreak detection and alerting are fast without compromising reliability, security, or data-privacy compliance. The accompanying traceability matrix confirms that every identified CI/CD requirement is fulfilled by a specific pipeline stage or mechanism.